Name: Andrik Brenth Canlas Section: BSIT 4-A

NPM output:

```
> shopsmart-starter@1.0.0 check
> npm run lint --workspace frontend && npm run lint --workspace backend && npm run test && npm run build

> shopsmart-frontend@1.0.0 lint
> eslint .

> shopsmart-backend@1.0.0 lint
> eslint .

> shopsmart-starter@1.0.0 test
> npm run test --workspace frontend && npm run test --workspace backend

> shopsmart-frontend@1.0.0 test
> vitest run

 ✓ src/utils/cart.test.js (7 tests) 3ms

 Test Files  1 passed (1)
      Tests  7 passed (7)

> shopsmart-backend@1.0.0 test
> node --test

✔ parseBudget extracts a peso budget
✔ rankProducts respects the budget
✔ assistant response contains traceable sources
✔ a valid classroom order passes validation
✔ duplicate products fail order validation
✔ an oversized assistant question fails validation
ℹ tests 6
ℹ pass 6

> shopsmart-starter@1.0.0 build
> npm run build --workspace frontend

> shopsmart-frontend@1.0.0 build
> vite build

✓ 52 modules transformed.
dist/index.html                  0.50 kB
```

dist/assets/index.css 4.54 kB
dist/assets/index.js 273.48 kB
✓ built in 903ms

Explanation:

When someone types in the search box, React sends a request to GET /api/products?search=<term>. On the server side, Express cleans up the search string and uses Prisma to find active products whose name or description matches. It converts any decimal prices into plain JavaScript numbers and sends back the array.

On the frontend, we handle rapid keystrokes using a simple active cleanup flag inside useEffect. If someone keeps typing or clears the box before an API call finishes, we drop the stale response so older data doesn't overwrite newer results. Once the fresh products arrive, client-side filtering and sorting handle the rest. We also keep the category dropdown populated from a persistent state list so options don't vanish as search results shrink.